SUMMER INTERNSHIP PROGRAM IN AI & ML · 2026 · PROJECT 1

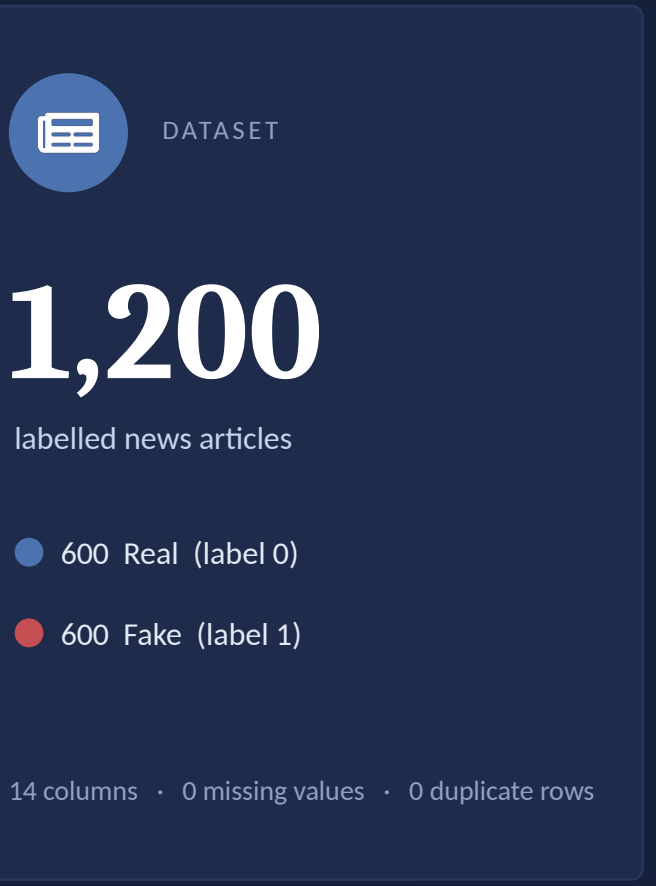
AI-Powered Fake News Detection

Using Text Classification

Natural Language Processing · Data Preparation Pipeline

Submitted by	Jahnvi Pandey · Reg. No. 605971
Guide	Dr. Ashok Gopalakrishnan
Programme	Bachelor of Computer Applications (BCA) — Semester IV
Institute	Indian Institute of Computing and Technology

Report reference: Fake_News_Detection_Project_Report.pdf · Notebook: Fake_News_Detection.ipynb



Introduction & Problem Statement

Why automated detection is needed, and what this project set out to solve

- News consumption has shifted to websites, aggregators and social feeds, where an article can be shared thousands of times within minutes of publication.
- Fabricated stories reach the same audience as verified reporting, often before any fact-checking organisation can respond.
- A fake article can look visually identical to a genuine one, so readers judge credibility by tone, headline phrasing or source name rather than by the underlying facts.
- NLP and Machine Learning offer a computational alternative: train a model to recognise the textual patterns — word choice, structure, sensationalism — statistically associated with fake content.

Problem Statement

Given a dataset of news articles labelled Real or Fake, prepare the raw textual data in a form suitable for machine learning, and lay the groundwork for a system that classifies unseen articles by analysing their content rather than their source alone.

Scope of this submission

Data loading, quality checks, text cleaning, tokenisation and word-count extraction are implemented. Vectorisation and model training are defined as the next phase.

Objectives

Five objectives were set at the start of the project (Report, Section 1.4)

Collect and examine the dataset

Load a labelled corpus of news articles suitable for binary Real / Fake classification.

ACHIEVED

Build a text-cleaning pipeline

Remove URLs, HTML tags, digits, punctuation and English stopwords from raw article text.

ACHIEVED

Tokenise and derive features

Split cleaned text into tokens and compute a word-count feature for each article.

ACHIEVED

Prepare data for vectorisation

Export a cleaned dataset ready for TF-IDF / Bag-of-Words and model training.

ACHIEVED

Document the pipeline

Record every step so training, evaluation and comparison can be added without rework.

ACHIEVED

All five objectives relate to the data-preparation phase and are complete. Model training and evaluation form the next phase of the project (Report, Chapters 5–6).

Dataset Description

GlobalFakeNews_Research2026_v1.csv · supplied locally · 1,200 rows × 14 columns

1,200
Records loaded

14
Columns in raw file

0
Missing values

0
Duplicate rows

Perfectly balanced target

label = 0 → Real (600) · label = 1 → Fake (600). No oversampling or class weighting is required during modelling.

Role	Columns
Identifier	id
Text fields	title, full_text
Metadata	source_name, publish_date, category, country
Target	label (0 = Real, 1 = Fake)
Auxiliary numeric	sentiment_score, clickbait_score, emotional_word_ratio, readability_score, source_trust_score
Pre-computed	text_embedding (vector stored as text)

The five auxiliary scores and text_embedding were not used anywhere in the current notebook. Only full_text and label enter the implemented pipeline.

Dataset Distribution

Class balance, news category and country of origin

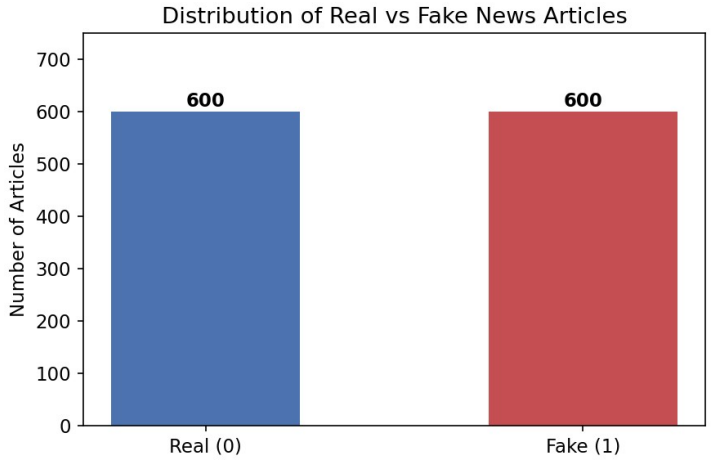


Figure 3.1 Real vs Fake articles

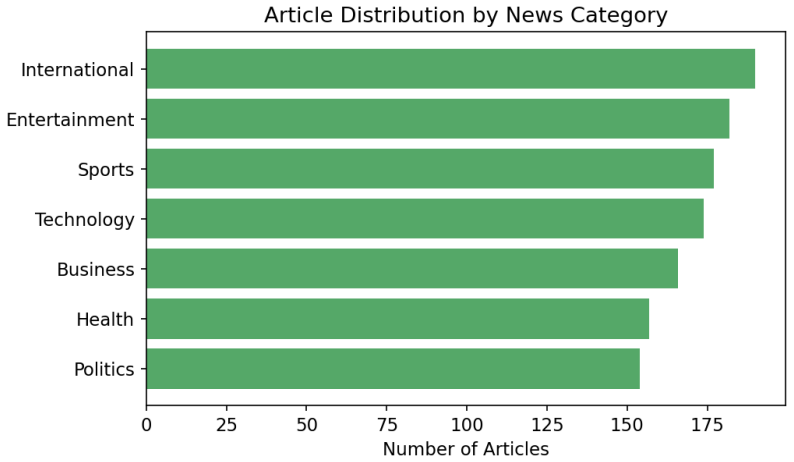


Figure 3.2 Articles by news category

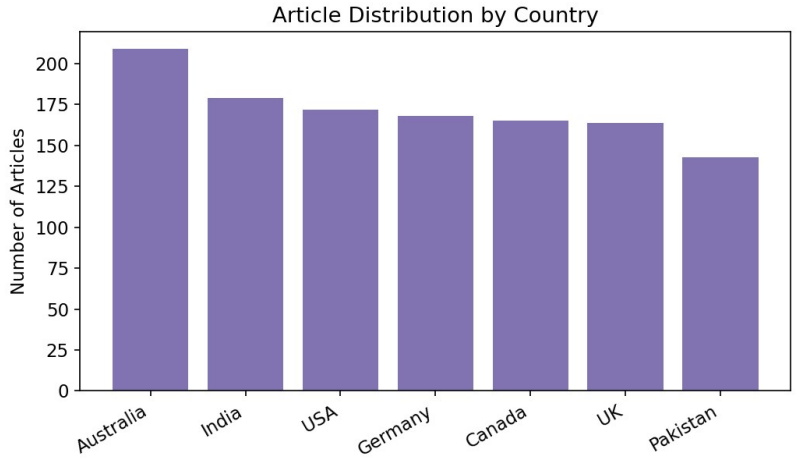


Figure 3.3 Articles by country

Balanced classes

600 Real and 600 Fake — no imbalance handling needed.

Seven categories

International (190) is largest, Politics (154) smallest.

Seven countries

Australia (209) and India (179) contribute the largest shares.

Figures reproduced from the project report (Chapter 3).

Data Preprocessing Pipeline

Seven stages implemented in the notebook, applied to the full_text column

Figure 4.1 Text Preprocessing Pipeline Implemented in the Notebook

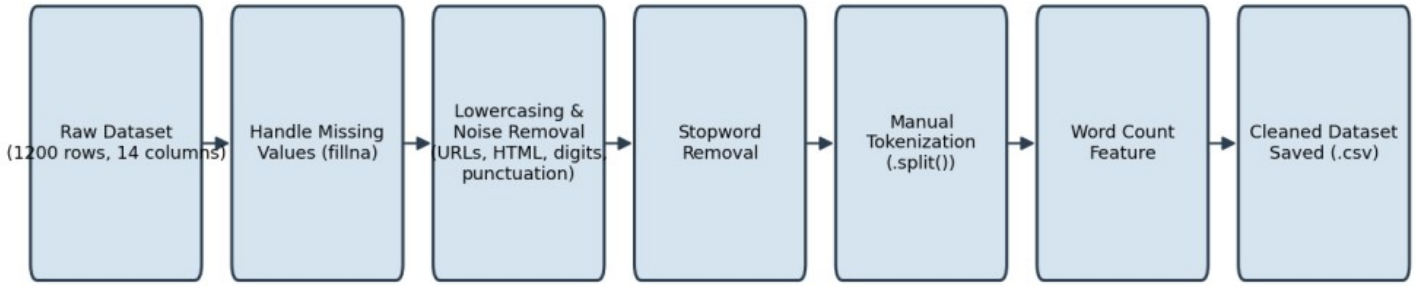


Figure 4.1 Text preprocessing pipeline implemented in the notebook

Tokenisation and feature derivation

- Tokenisation is manual — .split() on whitespace — which is safe because punctuation is already stripped.
- word_count is derived with len() on each token list, adding a numeric feature per article.
- clean_text() was written twice: an initial version, then the refined final version used for output.

clean_text() operations

Lowercase

```
str(text).lower()
```

Remove URLs

```
re.sub(r'http\S+', '')
```

Remove HTML

```
re.sub(r'<.*?>', '')
```

Remove digits

```
re.sub(r'\d+', '')
```

Remove punctuation

```
re.sub(r'[^\w\s]', '')
```

Collapse whitespace

```
re.sub(r'\s+', ' ').strip()
```

Remove stopwords

```
NLTK english corpus
```

Preprocessing Output

Effect of the cleaning function, and the resulting word-count feature

BEFORE full_text · 60 words

```
"This article provides detailed analysis based on verified
sources and official statements. This article provides detailed
analysis based on…"
```

AFTER clean_text · 45 tokens

```
"article provides detailed analysis based verified sources
official statements article provides detailed analysis based…"
```

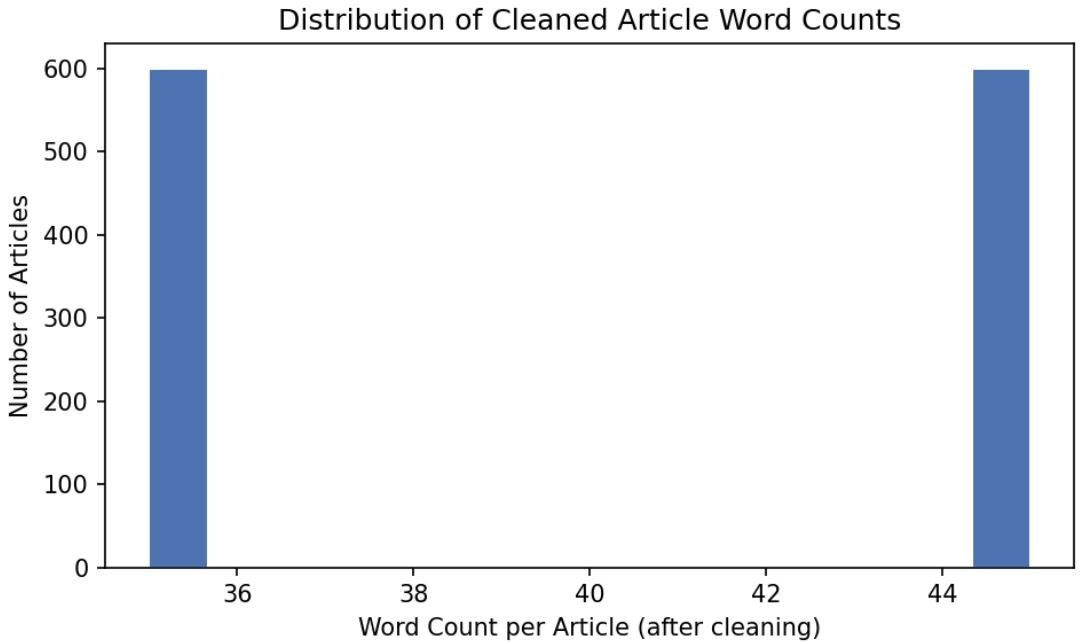


Figure 4.2 Distribution of cleaned article word counts

1,200 × 17
Final dataframe shape

+3 columns
clean_text, tokens, word_count

2 values
word_count takes only 45 or 35

Cleaned_Fake_News.csv
Exported cleaned dataset

Key Finding: The Dataset Is Trivially Separable

Analysis of the cleaned output revealed that several fields predict the label perfectly

2

unique article texts

All 1,200 records share just two distinct full_text values — one for every Real article, one for every Fake article.

100%

separable by length

word_count = 45 for all 600 Real articles and 35 for all 600 Fake articles, with no overlap.

10 / 10

sources map to labels

Each source_name belongs exclusively to one class (BBC, Reuters, CNN → Real; ViralNow24, InsiderBuzz → Fake).

3 of 5

scores never overlap

clickbait_score, emotional_word_ratio and source_trust_score have completely disjoint ranges per class.



What this means for the modelling phase

Any classifier trained on full_text will report near-perfect accuracy, but only because it is memorising two fixed template strings — not because it has learned to detect misinformation. The dataset is synthetically constructed, so results from it will not generalise to real news. Model comparison should therefore be reported alongside this caveat.

Model Development

PLANNED — NOT YET TRAINED

Four classifiers selected for the next phase (Report, Chapter 5)

K-Nearest Neighbours

NON-PARAMETRIC

Assigns a label from the majority class among the k closest articles in the vectorised feature space.

Simple baseline; scales poorly as the TF-IDF space grows, since distances are recomputed against the full training set.

Logistic Regression

PARAMETRIC

Models the probability of an article being fake with a sigmoid applied to a weighted sum of its features.

Trains quickly, performs well on sparse high-dimensional TF-IDF, and gives interpretable coefficients.

Random Forest

ENSEMBLE

Aggregates many decision trees, each on a bootstrap sample with a random feature subset, by majority vote.

Captures non-linear feature interactions; less directly interpretable and slower on wide text features.

Neural Network

DEEP LEARNING

Feed-forward network passing the article vector through dense hidden layers to a sigmoid output.

Can model complex interactions but needs more data and compute than the other three candidates.

All four models are to be trained on the same TF-IDF representation of clean_text using an identical train-test split, so their performance can be compared fairly. No model code exists in the submitted notebook, so no hyperparameters or results are reported here.

Evaluation Methodology

PLANNED — NO RESULTS YET

Metrics defined for the modelling phase (Report, Chapter 6)

1. **Accuracy**
   Proportion of articles correctly classified out of all articles evaluated.

2. **Precision**
   Of the articles predicted Fake, the proportion that truly are Fake — matters when false alarms are costly.

3. **Recall**
   Of the articles that truly are Fake, the proportion correctly identified — matters when missed fakes are costly.

4. **F1-score**
   Harmonic mean of precision and recall, used as a single balanced metric.

5. **Confusion Matrix**
   A 2 × 2 table revealing whether a model is biased toward one class.

Confusion matrix template

		Predicted: Real	Predicted: Fake
Actual	Real	TN	FP
	Fake	FN	TP

Cells are intentionally left unfilled — no models have been trained.

Reporting integrity

No accuracy, precision, recall or F1 figures appear in this presentation, because none have been produced.

Project Status & Limitations

What is implemented, what remains, and the constraints observed

Step	Operation	Status
1	Load dataset (pd.read_csv)	Implemented
2	Exploratory checks (shape, info, nulls, duplicates)	Implemented
3	Handle missing values (fillna)	Implemented
4	Lowercase, remove URLs / HTML / digits / punctuation	Implemented
5	Stopword removal	Implemented
6	Tokenisation (whitespace split)	Implemented
7	Word count feature	Implemented
8	Save cleaned dataset to CSV	Implemented
9	TF-IDF / Bag-of-Words vectorisation	Planned
10	Train-test split	Planned
11	Model training (KNN, LR, RF, NN)	Planned

Status table reproduced from the project report, Section 4.9.

Limitations observed

- The dataset is synthetically generated rather than scraped from real news, limiting how far any conclusion would generalise.
- Only two unique article texts exist, so full_text alone offers almost no lexical variety for a model to learn from.
- No vectorisation or model training is implemented, so no classification accuracy can be reported.
- clean_text() was rewritten once during development; only the final version feeds the saved output.
- The five auxiliary numeric scores and text_embedding remain unused by the current pipeline.

Conclusion & Future Scope

A tested data-preparation module, and a clear route to the classification stage

What was achieved

- A 1,200-record dataset was loaded and verified as complete — no missing values and no duplicate rows.
- A reproducible cleaning pipeline was implemented, covering lowercasing, URL, HTML, digit and punctuation removal, whitespace normalisation and stopword filtering.
- The text was tokenised and a word_count feature derived, producing a 1,200 × 17 dataframe exported as Cleaned_Fake_News.csv.
- Inspection of the cleaned output exposed a label leak in the dataset before any model was trained.
- Every preprocessing decision is documented, so vectorisation and modelling can be added without reworking the pipeline.

Future scope

1. Implement TF-IDF vectorisation on the clean_text column.
2. Perform the train-test split and train all four planned models.
3. Evaluate with accuracy, precision, recall, F1 and confusion matrices in one comparison table.
4. Combine the five auxiliary numeric scores with the text features, given the limited lexical variety.
5. Validate the pipeline on a real-world news dataset to test how well the approach generalises.

Thank you · Jahnvi Pandey · Reg. No. 605971 · Indian Institute of Computing and Technology